

第 10 章 非线性结构

数据结构可分为线性结构和非线性结构两种，上一章中主要讨论的是线性结构，一般来说，线性结构只能描述数据元素之间的线性顺序关系，而很难反映出数据元素之间存在的“非线性”关系。本章将讨论树和图两种非线性结构。

树形结构是一类非常重要的非线性结构，它用于描述数据元素之间的层次关系，树形结构在客观世界中广泛存在，例如，家族成员之间的血缘关系、公司的组织结构等都可以很自然地用树形结构来表示。树形结构中最常用到的是树和二叉树两种类型。本章先介绍树的定义、使用的术语，然后重点介绍二叉树的存储结构及其各种运算，并介绍树和森林与二叉树之间的转换关系以及二叉树的某些应用。

和树形结构一样图形结构也是一种非线性结构，在图中，数据元素之间的关系是任意的。图可以用来描述成千上万的实际问题，但我们只讨论其中的一小部分，在本章关于图形结构的内容中，不涉及图的数学性质，主要讨论图的存储结构和图的几个常用算法。

10.1 树的基本概念

树描述的是数据元素之间的层次关系。在树中，每一层上的数据元素可能和下一层中的多个元素相关，但最多只能和上一层中的一个元素相关。

树是 $n(n>0)$ 个数据元素的有限集合 T 。它满足以下两个条件：

- ①有且仅有一个特定的称为根的元素；
- ②其余元素分为 $m(m \geq 0)$ 个互不相交的有限集合 $T_1, T_2, \dots, T_m$ ，其中每个集合又都是一棵树并称其为根的子树。

树中的数据元素常称为结点。图 10.1 给出了一棵由 14 个结点组成的树。在最顶层的 A 为根结点，其余结点分成三个互不相交的有限集合 $T_1 = \{B, E, F, K, L\}$, $T_2 = \{C, G, M\}$, $T_3 = \{D, H, I, J, N\}$ 。 T_1, T_2, T_3 都是根 A 的子树，且它们本身又都是一棵树。

下面结合图 10.1 分别介绍树的一些常用术语。

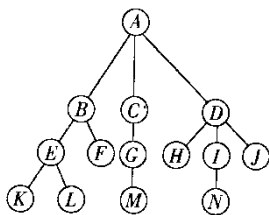


图 10.1 树

1) 双亲、子女、边 若结点 y 是结点 x 的一棵子树的根，则 x 称做 y 的“双亲”； y 称做 x 的“子女”；有序对 $\langle x, y \rangle$ 称做从 x 到 y 的“边”。例如，图 10.1 中， C 是 G 的双亲， G 是 C 的子女， $\langle C, G \rangle$ 是从 C 到 G 的边（它对应图中的有向线段 CG ）。

2) 兄弟 具有同一双亲的结点彼此称做“兄弟”。图 10.1 中， B, C, D 互为兄弟，而 F 和 G 不是兄弟。

3) 路径、路径长度 若树中存在着一个结点的序列 $k_1 k_2 \dots k_j$ ，使 k_i 是 k_{i+1} ($1 \leq i < j$) 的双亲，则称该结点序列为从 k_1 到 k_j 的一条路径；路径长度等于 $j-1$ ，它是该路径所经过的边的数目。例如，图 10.1 中， $ADIN$ 是从结点 A 到结点 N 的一条路径，其长度为 3。

4) 结点的层数 根结点的层数为 0，其余结点的层数等于其双亲结点的层数加 1。在图 10.1 中，层数为 0 的结点有 A ，层数为 1 的结点有 B, C, D ，等等。

5) 树的深度（高度） 即树中层数最大的结点的层数。图 10.1 所示树的深度为 3。

6) 结点的度数、树的度数 一个结点子女的个数称为该结点的“度数”。树中度数最大的结点的度数叫做“树的度数”。例如, 图 10.1 中, A、C、E、J 的度数分别为 3、1、2、0; 树的度数为 3。

7) 树叶、分支结点 度数为 0 的结点叫做“树叶”(又叫终端结点); 度数大于 0 的结点叫做“分支结点”或“内结点”。在图 10.1 中, K、L、F、M、H、N、J 都是树叶, 其余结点都是分支结点。

8) 有序树、无序树 若将树中每个结点的各个子树看成是从左到右有序的(即不能互换), 则称该树为有序树; 否则为无序树。以下仅讨论有序树。

9) 森林 m ($m \geq 0$) 棵互不相交的树的集合称为森林, 如图 10.2 所示。

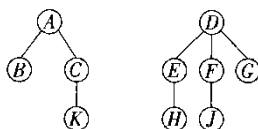


图 10.2 森林

10.2 二叉树

二叉树是一种特殊的树形结构, 它有一些极其有用的性质, 与树相比, 二叉树在结构上更规范和更有确定性, 因此, 对二叉树的存储和运算比树更加简单。此外, 在树或森林与二叉树之间有一种简单的转换关系, 可以把一棵树或一个森林唯一地转换为一棵二叉树, 也可以把一棵二叉树唯一地转换为一棵树或一个森林, 而且这两种转换是可逆的。所以, 对树的处理常用二叉树实现。

10.2.1 二叉树的定义

二叉树是 n ($n \geq 0$) 个结点的有限集合。它或者是空集 ($n = 0$), 或者由一个根结点及两棵互不相交的、分别称为这个根的左子树和右子树的二叉树组成。

由二叉树的定义可知, 二叉树有五种基本形态(图 10.3)。

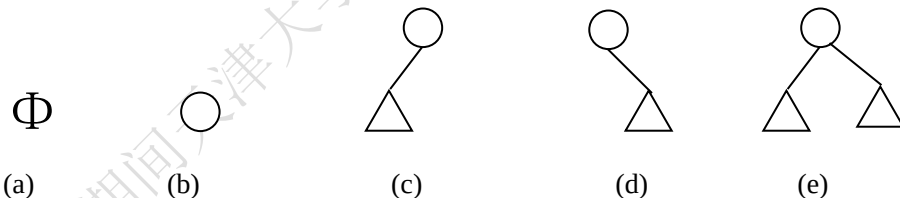


图 10.3 二叉树的五种基本形态

(a) 空二叉树; (b) 仅有一个根结点的二叉树; (c) 右子树为空的二叉树;
(d) 左子树为空的二叉树; (e) 左、右子树非空的二叉树

需要指出的是, 二叉树并不是树的特殊形式。尽管两者有许多相似之处, 使用的术语也基本相通, 但它们是两种不同的数据结构。树和二叉树之间最主要的差别在于, 二叉树结点的子树有左、右之分, 即使在结点仅有一棵子树的情况下, 也要明确指出该子树是左子树还是右子树。例如, 图 10.3 的 (c) 和 (d) 是两棵不同的二叉树, 但如果作为树, 它们是相同的。此外, 二叉树可以是空, 而树不能为空。

10.2.2 二叉树的性质

二叉树具有以下重要性质。

性质 1 二叉树第 i 层上的结点数最多为 2^i ($i \geq 0$)。

性质 2 深度为 k 的二叉树至多有 $2^{k+1}-1$ 个结点 ($k \geq 0$)。

以上两个性质的证明并不复杂, 此处从略。

性质 3 在任意一棵二叉树中, 若终端结点的个数为 n_0 、度数为 2 的结点的个数为 n_2 , 则 $n_0 = n_2 + 1$ 。

证明 因为二叉树中所有结点的度数均不大于 2, 所以结点总数 (记为 n) 应等于度数为 0 的结点的个数 n_0 、度数为 1 的结点的个数 n_1 和度数为 2 的结点的个数 n_2 之和, 即

$$n = n_0 + n_1 + n_2 \quad (10-1)$$

设二叉树边的个数为 b , 由于二叉树中除了根结点外, 其余结点都有且仅有一条与其双亲相连的边, 所以

$$b = n - 1$$

同时, 二叉树中与度数为 1 的结点相连的边有一条, 与度数为 2 的结点相连的边有两条, 所以又有

$$b = n_1 + 2n_2$$

于是得

$$n = n_1 + 2n_2 + 1 \quad (10-2)$$

由式(10-1)和式(10-2)得

$$n_0 = n_2 + 1$$

以上三个性质是所有二叉树所共有的。二叉树的其他性质只与某些特殊的二叉树相关, 为此, 先定义两种特殊的二叉树。

定义一(满二叉树) 一个深度为 k ($k \geq 0$) 且有 $2^{k+1}-1$ 个结点的二叉树称为满二叉树。

图 10.4 (a) 是一棵深度为 3 的满二叉树。

定义二(完全二叉树) 若一棵二叉树至多只有最下面两层上的结点的度数可以小于 2, 并且最下面一层上的结点都集中在该层最左边的若干位置上, 则称此二叉树为完全二叉树。

图 10.4(b) 是一棵完全二叉树。显然, 满二叉树是完全二叉树, 但完全二叉树不一定是满二叉树。在满二叉树最下一层上, 从最右端开始向左连续删除若干个结点后得到的一定是一棵完全二叉树。若删除的结点是不连续的, 得到的就不是一棵完全二叉树 (图 10.4(c))。

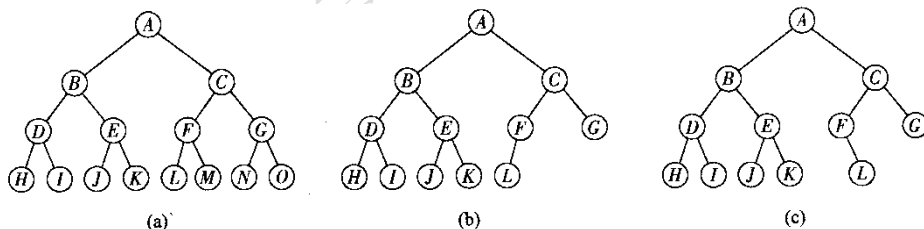


图 10.4 特殊形态的二叉树

(a) 满二叉树; (b) 完全二叉树; (c) 非完全二叉树

完全二叉树具有以下性质。

性质 4 具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor$ 。

证明 设深度为 k 的完全二叉树具有 n 个结点, 则根据性质 2 和完全二叉树的定义有:

$$2^k - 1 < n \leq 2^{k+1} - 1$$

即

$$2^k \leq n < 2^{k+1}$$

取对数后有

$$k \leq \log_2 n < k+1$$

因 k 为整数, 故有 $k = \lfloor \log_2 n \rfloor$ 。

性质 5 若对一棵有 n 个结点的完全二叉树，按自顶向下、同一层由左到右的顺序依次为其每个结点编号（设编号从 0 开始，如图 10.5 所示），则对编号为 i 的结点 k_i ($0 \leq i \leq n-1$) 有以下关系成立：

- ①若 $i > 0$ ，则 k_i 双亲结点的编号为 $\lfloor (i-1)/2 \rfloor$ ；
- ②若 $i = 0$ ，则 k_i 是根结点，它没有双亲；
- ③若 $2i+1 < n$ ，则 k_i 左子女结点的编号是 $2i+1$ ，否则 k_i 无左子女，即它是一个叶子；
- ④若 $2i+2 < n$ ，则 k_i 右子女结点的编号为 $2i+2$ ，否则 k_i 无右子女。

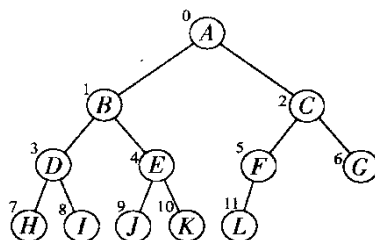


图 10.5 结点编号的完全二叉树

10.2.3 二叉树的存储结构

1. 顺序存储结构

要想实现二叉树的顺序存储，就必须把二叉树中的结点安排成一个线性序列，且使得结点在这个序列中的相对位置能够反映出结点之间的逻辑关系。对结点编号的完全二叉树，如果将其所有的结点按编号顺序依次存储在一个一维数组里，则每个结点的编号与其在一维数组中对应的元素下标相一致，利用性质 5，可以找出任何一个结点的双亲、子女等结点在一维数组中的位置。图 10.6 是图 10.4(b)所示完全二叉树的顺序存储结构。

0	1	2	3	4	5	6	7	8	9	10	11
A	B	C	D	E	F	G	H	I	J	K	L

图 10.6 完全二叉树的顺序存储结构

用以上顺序存储结构存储完全二叉树既简单又节省存储空间，但对一般的二叉树而言，当采取顺序存储时，为了能用结点在一维数组中的相对位置表示结点之间的逻辑联系，也必须按照完全二叉树的形式存储树中的结点。例如，为实现如图 10.7 (a) 所示的非完全二叉树的顺序存储，需要加上一些实际并不存在的“虚结点”，使它成为图 10.7 (b) 所示的完全二叉树，相应的顺序存储结构如图 10.7 (c) 所示。如果需要加入的“虚结点”比较多，会造成存储空间的浪费，所以，在实际应用中，对非完全二叉树一般不采用上述的顺序存储方式。

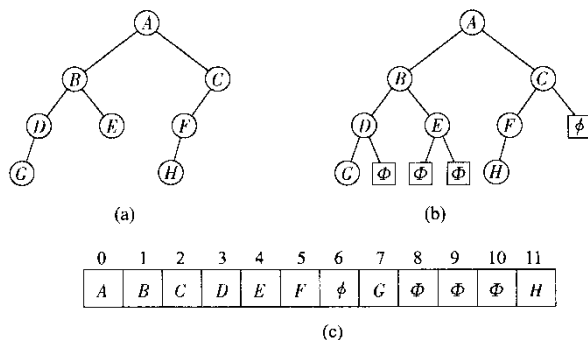


图 10.7 二叉树的顺序存储结构

(a) 二叉树；(b) 完全二叉树；(c) 顺序存储结构

2. 链式存储结构

存储二叉树最直接的方法是链接存储。由于二叉树的每个结点最多有两个子女，因此，用链接方法存储二叉树时，每个结点中除存储结点的自身值外，还应设置两个指针域 lchild 和 rchild。它们分别指向结点的左子女和右子女，结点结构如图 10.8 所示。

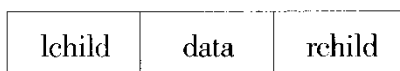


图 10.8 链接存储二叉树时结点的结构

一棵二叉树中所有这样结构的结点再加上一个指向根结点的指针 root，就构成了二叉树的链接存储表示。图 10.9 给出了图 10.7 (a) 所示的二叉树的链接表示。

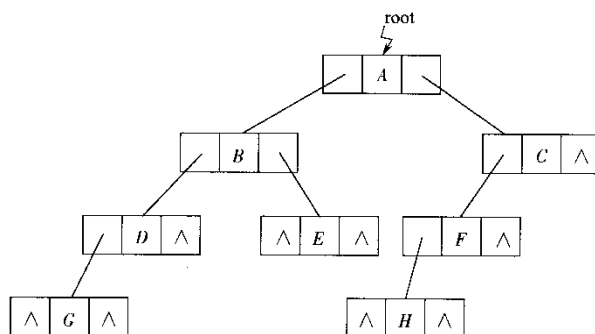


图 10.9 二叉树的链接存储结构

下面给出了二叉树结点类 Node 的定义。在此假定二叉树结点的数据域为整型(int 型)。有关二叉树类 BinTree 的定义见 10.2.5 节的例 10.2。

例 10.1 整型二叉树结点类的定义。

```
class BinTree;           //二叉树类 BinTree 的前视声明
class Node{
    friend class BinTree; //定义二叉树类 BinTree 为友元
public:
    Node():lchild(0),rchild(0){} //无参构造函数
    Node(int val, Node *lptr=0, Node *rptr=0){ //带参构造函数
        data=val;
        lchild=lptr;
        rchild=rptr;
    }
    int Getdata(){return data;} //返回结点数据
    Node *Left(){return lchild;} //返回左子女指针
    Node *Right(){return rchild;} //返回右子女指针
private:
    Node *lchild,*rchild;
    int data;
};
```

类 Node 中有两个构造函数，第一个构造函数建立了一个 lchild 和 rchild 皆为空的结点，第二个构造函数建立一个叶或非叶结点并用给定的值 val 初始化该结点的 data 字段。

10.2.4 二叉树的遍历

遍历二叉树是二叉树的一种重要运算。所谓遍历二叉树是指沿着某条搜索路线周游二叉树，对树中的每个结点进行访问而且只访问一次。遍历一个线性结构很容易，只要从开始结

点出发顺序扫描每个结点即可。但二叉树是一种非线性结构,每个结点可以有两个直接后继,因此,需要寻找一种规律来系统地访问二叉树中的各个结点。

由二叉树的定义可知,一棵非空的二叉树由根结点、左子树和右子树三个基本部分组成。因此,对一棵非空的二叉树的遍历可分解为对其根结点、左子树、右子树的遍历。如果用 D、L、R 分别表示一棵非空的二叉树的根、左子树和右子树,则对二叉树的遍历有 DLR、LDR、LRD、DRL、RDL、RLD 六种方案。其中,前三种方案是按先左后右的次序遍历根的两棵子树,而后三种方案则是前三种方案的镜像,为方便起见,下面只讨论前三种方案。三种遍历方案的递归定义如下。

1) 先序遍历 (DRL 次序) 若二叉树非空,则依次进行如下操作:访问根结点,先序遍历左子树,先序遍历右子树。

2) 中序遍历 (LDR 次序) 若二叉树非空,则依次进行如下操作:中序遍历左子树,访问根结点,中序遍历右子树。

3) 后序遍历 (LRD 次序) 若二叉树非空,则依次进行如下操作:后序遍历左子树,后序遍历右子树,访问根结点。

除上述三种遍历二叉树的方法外,还可以按层次遍历二叉树

4) 层次遍历 按层数由小到大、同一层从左到右顺序依次访问二叉树的各个结点。

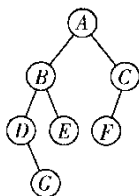


图 10.10 二叉树

对图 10.10 所示二叉树进行先序、中序、后序和层次遍历后得到的结点访问序列依次为:

先序访问序列 ABDGECF

中序访问序列 DGBE AFC

后序访问序列 GDBE AFC

层次访问序列 ABCDEFG

10.2.5 二叉树类

按前述链接方式存储二叉树,在 10.2.3 节例 10.1 定义的二叉树结点类 Node 的基础上,可定义如下的二叉树类 Bintree。

例 10.2 整型二叉树类的定义。

```

#include <iostream.h>

class BinTree{
public:
    BinTree():root(0){}                //构造一棵空树
    ~BinTree(){Destroy(root);}          //析构函数
    void insertNode(int val){            //向二叉树插入值为 val 的结点
        insertNode(root,val);
    }
    void PreOrder(){PreOrder(root);}     //先序遍历二叉树
    void InOrder(){InOrder(root);}       //中序遍历二叉树
    void PostOrder(){PostOrder(root);}   //后序遍历二叉树
private:
    Node *root;                          //根结点指针
  
```

```

void insertNode(Node *&t,int val);    //向 t 指向的二叉树中插入结点
void PreOrder(Node *t);              //先序遍历 t 指向的二叉树
void InOrder(Node *t);               //中序遍历 t 指向的二叉树
void PostOrder(Node *t);             //后序遍历 t 指向的二叉树
void Destroy(Node *t);               //删除二叉树

```

```
};
```

类 BinTree 的私有成员函数 insertNode()的功能是,按照二叉排序树的特点向以 t 所指向的结点为根的二叉树中插入一个结点值为 val 的新结点。二叉排序树是一种特殊的二叉树。其特点是:左子树上所有结点的值均小于其双亲结点的值,右子树上所有结点的值均大于或等于其双亲结点的值(关于二叉排序树的详细情况见 11.1.3 节的讨论)。在二叉排序树中插入值为 val 的结点时,首先要找到插入位置,当 t 不为空时,用 val 与 t 所指向结点的值进行比较,如果前者小于后者,则进入 t 所指向结点的左子树,否则进入右子树。当 t 为空时,找到插入位置,实施插入。注意,如果插入成功,与函数第一个形式参数相结合的实际参数(它应该是指向二叉树根结点的指针变量或新插入结点双亲的 lchild 或 rchild)中必须存放新插入结点的地址。为实现这种改变,函数的第一个形参被定义为指针变量的引用。

```

void BinTree::insertNode(Node *&t,int val){
    if(t==0) t=new Node (val);
    else if(val<t->data) insertNode(t->lchild,val);
    else insertNode(t->rchild,val);
}

```

下面给出 BinTree 类中实现对以 t 指向的二叉树进行先序、中序和后序遍历的三个私有成员函数的定义,它们均采用了递归的实现方式。语句如下:

```

void BinTree::PreOrder(Node *t){
    if(t!=0){
        cout<<t->data<<" ";    //访问根结点
        PreOrder(t->lchild);    //先序遍历左子树
        PreOrder(t->rchild);    //先序遍历右子树
    }
}

void BinTree::InOrder(Node *t){
    if(t!=0){
        InOrder(t->lchild);    //中序遍历左子树
        cout<<t->data<<" ";    //访问根结点
        InOrder(t->rchild);    //中序遍历右子树
    }
}

void BinTree::PostOrder(Node *t){
    if(t!=0){
        PostOrder(t->lchild);    //后序遍历左子树
        PostOrder(t->rchild);    //后序遍历右子树
        cout<<t->data<<" ";    //访问根结点
    }
}

```

Destroy()成员函数用于释放由 t 所指向的二叉树中的所有结点:

```

void BinTree::Destroy(Node *t){
    if(t!=0){
        Destroy(t->lchild);
        Destroy(t->rchild);
        delete t;
    }
}

```

为方便使用，可以把 10.2.3 中给出的结点类 Node 和本节给出的二叉树类 BinTree 组织到文件 bintree.h 中。下面是一个测试二叉树类的程序。

例 10.3 整型二叉树类测试程序。

```

#include "bintree.h"
void main(){
    BinTree BTree;
    int x;
    cout<<"请输入 10 个整数: \n";
    for(int i=0;i<10;i++){
        cin>>x;
        BTree.insertNode(x);
    }
    cout<<"先序遍历序列: \n";
    BTree.PreOrder();
    cout<<"\n 中序遍历序列: \n";
    BTree.InOrder();
    cout<<"\n 后序遍历序列: \n";
    BTree.PostOrder();
    cout<<endl;
}

```

程序运行结果：

```

请输入 10 个整数:
47 23 56 15 26 89 12 61 55 31
先序遍遍历序列:
47 23 15 12 26 31 56 55 89 61
中序遍历序列:
12 15 23 26 31 47 55 56 61 89
后序遍历序列:
12 15 31 26 23 55 61 89 56 47

```

10.2.6 树、森林与二叉树的转换

在树或森林与二叉树之间有一种自然的一一对应关系，任何一棵树或一个森林都能唯一地对应一棵二叉树；反之亦然。

1. 树、森林到二叉树的转换

把树转换成二叉树的方法是：

- ①在所有的兄弟之间加一条连线；
- ②对每一个结点，除了保留与最左边的子女的连线外，去掉与其他子女的连线；
- ③将保留下来的边作为左子树的边，兄弟间的连线作为右子树的边。

图 10.11(a)所示的树经①、②步处理后变为图 10.11(b)的形式,再经第③步的整理,即可得到转换后的二叉树(图 10.11(c))。由于树根没有兄弟,所以转换后得到的二叉树的根结点的右子树必为空。

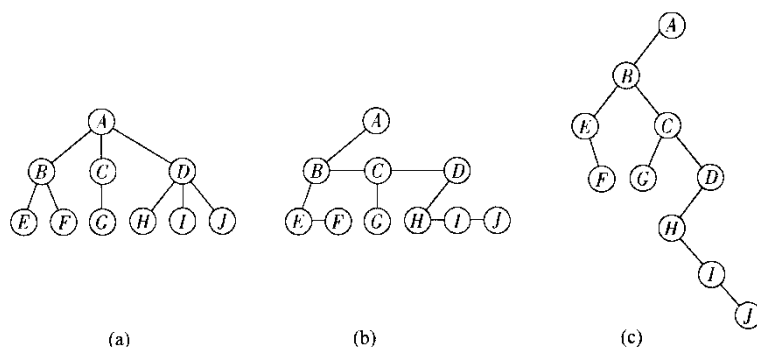


图 10.11 树转换为二叉树

(a) 树; (b) 中间形式; (c) 转换结果

将一个森林转换成二叉树的方法是:先将森林中的每一棵树变为二叉树,然后将各二叉树的根结点看成兄弟,用线把它们连到一起,经整理后可得到相应的二叉树。图 10.12(b)是图 10.12(a)所示森林转换后得到的结果,图 10.12(c)是经整理后得到的二叉树。

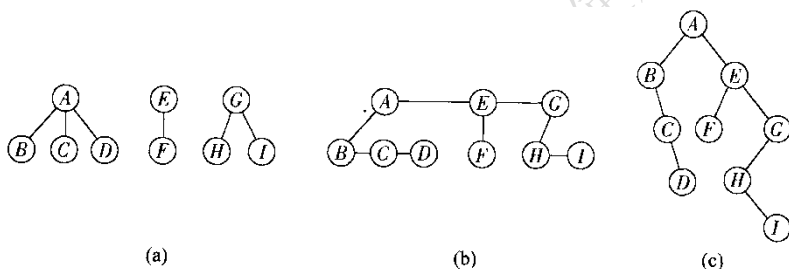


图 10.12 森林到二叉树的转换

森林转换为二叉树的形式定义是: 设 $F=\{T_1, T_2, \dots, T_n\}$ 是一个由树 $T_1, T_2, \dots, T_n$ 组成的森林, 则森林 F 对应的二叉树 $B(T_1, T_2, \dots, T_n)$ (也记作 $B(F)$) 如下:

- ①若 F 为空 ($n=0$), 则 $B(F)$ 为空二叉树;
- ②若 F 非空 ($n>0$), 则 $B(F)$ 的根就是 T_1 的根 W_1 , $B(F)$ 的左子树是 $B(T_{11}, T_{12}, \dots, T_{1m})$ (其中 $T_{11}, T_{12}, \dots, T_{1m}$ 是 W_1 的子树), $B(F)$ 的右子树是 $B(T_2, \dots, T_n)$ 。

2. 二叉树到树、森林的转换

把二叉树转换成森林也有一种自然的方法: 若结点 x 是其双亲 y 的左子女, 则把 x 的右子女、右子女的右子女……都与 y 连线, 最后去掉所有双亲到右子女的连线。图 10.13(b)是用上述方法对图 10.13(a)所示二叉树进行处理后得到的森林。

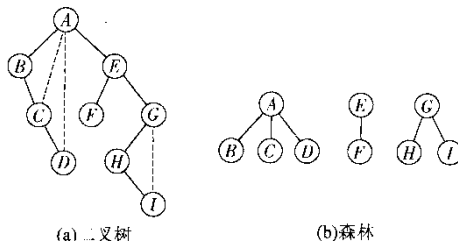


图 10.13 二叉树到森林的转换

以上转换方法的形式定义是: 设 B 是一棵二叉树, r 是 B 的根, L 是 r 的左子树, R 是 r 的右子树, 则对应于 B 的森林 $F(B)$ 的定义是:

- ①若 B 为空, 则 $F(B)$ 是空的森林;

②若 B 不为空, 则 $F(B)$ 是一棵树 T_1 加上森林 $F(R)$, 其中树 T_1 的根为 r , r 的子树为 $F(L)$ 。

10.2.7 哈夫曼树

1. 哈夫曼树的概念

(1) 扩充二叉树和带权路径长度

假设 $\{W_0, W_1, \dots, W_{n-1}\}$ 是 n 个实数的集合, 其中 $W_i \geq 0$ ($0 \leq i \leq n-1$)。若 T 是一棵有 n 个叶结点的二叉树, 而且将 $W_0, W_1, \dots, W_{n-1}$ 分别赋给 T 的 n 个叶结点作为它们的权, 则称 T 是权值为 $W_0, W_1, \dots, W_{n-1}$ 的扩充二叉树。带有权值的叶结点叫做扩充二叉树的外结点, 其余的分支结点叫做内结点。一个有 n 个外结点的扩充二叉树的带权路径长度 (WPL) 为:

$$WPL = \sum_{i=0}^{n-1} W_i l_i$$

其中, W_i 为外结点 i 所带的权值; l_i 为从根结点到外结点 i 的路径长度。

例如, 对图 10.14(a)所示的扩充二叉树, 其带权路径长度为

$$WPL = 2 \times 2 + 4 \times 2 + 6 \times 2 + 8 \times 2 = 40$$

根据给定的一组权值, 可以构造出具有不同带权路径长度的扩充二叉树。例如, 图 10.14 中的三棵二叉树都是权值为 $\{6, 8, 4, 2\}$ 的扩充二叉树。它们的 WPL 分别是 40、50 和 38。

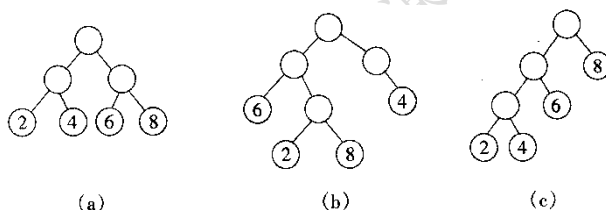


图 10.14 具有不同带权路径长度的扩充二叉树

(a) WPL=40

(b) WPL=50

(c) WPL=38

(2) 最优二叉树

通常, 把权值取为 $\{W_0, W_1, \dots, W_{n-1}\}$ 的所有扩充二叉树中 WPL 为最小的扩充二叉树称为最优二叉树。

(3) 哈夫曼树

利用哈夫曼提出的方法构造出的最优二叉树称为哈夫曼树。

2. 哈夫曼树的构造

哈夫曼树的构造方法如下。

①由给定的 n 个权值 $\{W_0, W_1, \dots, W_{n-1}\}$ 构造含有 n 棵扩充二叉树的森林 F , 森林中的每棵二叉树都只有一个根结点, 且每个根结点都取一个各不相同的 W_i 作为权值;

②用森林 F 中根结点的权值为最小和次小的两棵二叉树作为左、右子树构造出一棵新的二叉树, 并将新二叉树的根结点的权值取为左、右子树根结点权值之和;

③从森林 F 中删去作为新二叉树左、右子树的两棵二叉树, 将新构造的二叉树加入到森林 F 中;

④重复步骤②和③, 直到 F 中仅剩下一棵二叉树为止。

最后剩下的这棵二叉树就是哈夫曼树。

设给定一组权值 $\{6, 2, 3, 3, 4, 9\}$, 用哈夫曼算法构造哈夫曼树的过程如图 10.15 所示。图中, 带权的外结点用圆形框表示, 内结点用矩形框表示。

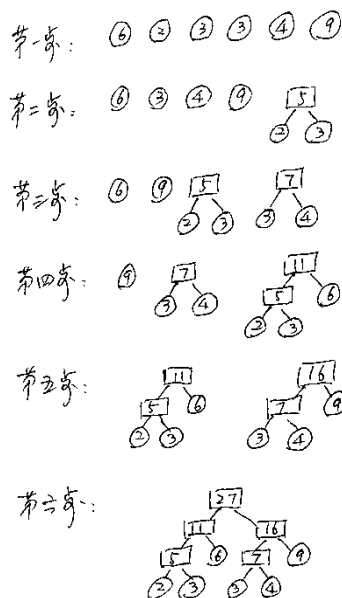


图 10.15 哈夫曼树的构造过程

由哈夫曼树的构造方法可知，在一棵哈夫曼树中不存在度数为 1 的结点。

3. 哈夫曼树的应用

哈夫曼树的应用十分广泛，在不同的应用中，对叶结点的权值也有不同的解释。例如，当把哈夫曼树应用到信息编码时，可将叶结点的权值看成是某个符号使用的频率；应用到判定过程时，权值可以看成是某个数据出现的频率，等等。下面介绍哈夫曼树的一种重要应用——哈夫曼编码。

在数据通信中，发送端需要将发送电文中的字符转换成由二进制数字 0、1 组成的序列（称为编码），而在接收端则需要将收到二进制数还原成原来的字符（称为译码）。为节省时间，总希望发送的电文代码尽可能地短些。如果能按电文中各字符出现的频度，使出现频度较少的字符采用位数较多的编码，而出现频度较大的字符采用位数较少的编码，则可以使电文的代码长度减至最少。当然，在采用这种不等长编码时还必须保证任何一个字符的编码都不能是另外一个字符编码的前缀，否则就会出现译码不唯一的情况，例如，设字母 a, b, c, d 的编码分别是：0, 1, 00 和 01，则二进制串“010001”可以被译为 abaaab、dcd、abcd 等等。利用哈夫曼树可以得到满足以上两个要求的编码，即哈夫曼编码。

设 $C = \{C_0, C_1, \dots, C_{n-1}\}$ 是需要进行编码的字符集， $W = \{W_0, W_1, \dots, W_{n-1}\}$ 是每个字符在电文中出现的频度，以 W_i 为权值构造出一棵哈夫曼树，并为指向左子女的边规定编码为 0，为指向右子女的边规定编码为 1。于是，从根结点到权值为 W_i 的外结点（对应于字母 C_i ）的路径上所经过的边的编码构成了一个由 0 和 1 组成的序列。此序列即为 C_i 的哈夫曼编码。

例如，设 $C = \{A, B, C, D, E, F\}$ ，各字符出现的频率为 $\{6, 2, 3, 3, 4, 9\}$ ，利用 $\{6, 2, 3, 3, 4, 9\}$ 构造的哈夫曼树如图 10.15 所示。各字符的哈夫曼编码是：A=01, B=000, C=001, D=100, E=101, F=11。

可以看出，采用哈夫曼编码可以使给定的电文编码达到最短，而且，由于在哈夫曼树中每个字符对应的结点都是叶结点，而一个叶结点不可能在从根结点到其他叶结点的路径上，因此，任何一个字符的哈夫曼编码不可能是另一个字符的哈夫曼编码的前缀，这样就保证了译码的唯一性。

10.3 图(略)

10.4 例题

例 10.4 利用二叉树的遍历可以实现多种关于二叉树的运算，本例将介绍计算二叉树深度、求二叉树结点个数和求二叉树叶结点个数的算法。

(1) 计算二叉树的深度

通过后序遍历二叉树的，可先得到左子树的深度 dl ，然后再得到右子树的深度 dr ，显然，二叉树的深度应为 $1+\max(dl,dr)$ 。设空二叉树的深度为-1。

为了能够通过二叉树类的对象调用求深度的函数 `Depth()`，可在 10.2.5 节给出的二叉树类 `BinTree` 的 `publis` 部分增加以下函数：

```
int Depth(){return Depth(root);}
```

在 `private` 部分增加：

```
int Depth(Node *t);          //返回以 t 指向的结点为根的子树的深度
```

该私有成员函数的实现为：

```
int BinTree::Depth(Node *t){
    if(t==0) return -1;
    int dl=Depth(t->lchild);
    int dr=Depth(t->rchild);
    return 1+(dl>dr?dl:dr);
}
```

(2) 求二叉树结点个数

在遍历二叉树时，对每个结点都要访问一次，只要计算出在遍历过程中访问的结点数目，也就求出了二叉数中结点的个数。下面说明求二叉树结点个数的具体过程。

在 10.2.5 节给出的二叉树类 `BinTree` 的 `publis` 部分增加以下函数：

```
int Size(){return Size(root);}
```

在 `private` 部分增加：

```
int Size(Node *t);          //返回以 t 指向的结点为根的子树中结点的个数
```

该私有成员函数的实现为：

```
int BinTree::Size(Node *t){
    if(t==0)return 0;
    return 1+Size(t->lchild)+Size(t->rchild);
}
```

(3) 求二叉树叶结点个数

在遍历二叉树时，若发现某个结点的左、右子树皆为空，则该结点是叶子。求二叉树叶结点个数的过程如下。

在 10.2.5 节例 10.2 给出的二叉树类 `BinTree` 的 `publis` 部分增加以下函数：

```
int LeafCount(){return LeafCount(root);}
```

在 `private` 部分增加：

```
int LeafCount(Node *t);      //返回以 t 指向的结点为根的子树中叶结点的个数
```

该私有成员函数的实现为：

```
int BinTree::LeafCount(Node *t){
    if(t==0)return 0;
    if(t->lchild==0 && t->rchild==0) return 1;
    else return LeafCount(t->lchild)+LeafCount(t->rchild);
}
```

下面是对以上函数进行测试的程序。

```
#include "bintree.h"
void main(){
    BinTree intBTree;
    int x[12]={47,23,56,15,26,89,12,61,55,31,12,99};
    for(int i=0;i<12;i++) intBTree.insertNode(x[i]);
    cout<<"二叉树的深度为: "<<intBTree.Depth()<<endl;
    cout<<"二叉树中结点的个数为: "<<intBTree.Size()<<endl;
    cout<<"二叉树中叶结点的个数为: "<<intBTree.LeafCount()<<endl;
}
```

仅供疫情期间天津大学师生学习课程使用，严禁翻印！

练习 10

一、单项选择题

- 如果结点 B 有 3 个兄弟，其双亲结点为 A，则 A 的度是 ____。
A) 2 B) 3 C) 4 D) 5
- 对于一个具有 n 个结点的二叉树，当它为 ____ 时高度最大。
A) 满二叉树 B) 单支树 C) 完全二叉树 D) 没有右子树
- 当以链接方式存储一个有 n 个结点的二叉树时，其二叉链表中有 ____ 个指针域为空。
A) 0 B) $n-1$ C) $n+1$ D) n
- 在一棵有 18 个结点的完全二叉树中，编号为 7 的结点的右子女的编号为 ____。
A) 15 B) 16 C) 17 D) 无右子女
- 在一棵非空二叉树的中序遍历序列中，根结点的右边只包含 ____。
A) 左子树上的部分结点 B) 左子树上的全部结点
C) 右子树上的部分结点 D) 右子树上的全部结点
- 根据使用频率，为 5 个字符设计的哈夫曼编码，以下不可能的编码是 ____。
A) 000, 001, 01, 10, 11 B) 111, 110, 101, 100, 0
C) 011, 000, 01, 10, 11 D) 110, 111, 01, 00, 10
- 在有 n 个顶点的有向图中，每个顶点的度数最大可达 ____。
A) $n-1$ B) $2(n-1)$ C) n D) $n+1$
- 一个具有 10 个顶点的无向图，边的总数最多为 ____。
A) 9 B) 90 C) 45 D) 18
- 设用邻接矩阵法存储一个有向图，若要删除第 i 个顶点的所有出边，只需将 ____。
A) 矩阵的第 i 行全部置为 0 B) 矩阵的第 i 行全部置为 1
C) 矩阵的第 i 列全部置为 0 D) 矩阵的第 i 列全部置为 1
- 以下说法中正确的是 ____。
A) 任何 AOV 网的拓扑排序的结果是唯一的
B) 对任何 AOV 网都可求出拓扑序列
C) 在拓扑序列中顶点的排列顺序是任意的
D) 拓扑序列表示了活动之间的先后关系

二、简答题

- 试画出具有 3 个结点的树和具有 3 个结点的二叉树的所有不同形态。
- 已知一棵度为 m 的树中，有 n_1 个度为 1 的结点， n_2 个度为 2 的结点，……， n_m 个度为 m 的结点，请推导出它有多少个叶结点？
- 写出图 10.36 所示的二叉树的叶结点、分支结点和每个结点的层数。该二叉树的深度是多少？

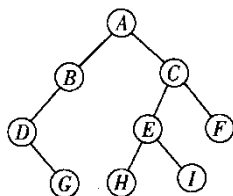


图 10.36 二叉树

- 当以顺序方式存储图 10.36 所示的二叉树时，需要增加多少个“虚结点”？
- 对图 10.36 所示的二叉树分别写出其先序、中序、后序和层次遍历序列。
- 设完全二叉树有 101 个结点，按结点的层次顺序把各结点存储在数组 $T[0...100]$ 中，对某

个 $k(0 \leq k \leq 100)$, 若 k 是叶结点, 则 k 的最小值是多少?

7. 对二叉树 T 中的一个结点 x , 它在先序序列、中序序列和后序序列中的序号分别用 $\text{pre}(x)$ 、 $\text{in}(x)$ 和 $\text{post}(x)$ 表示, 设 a 和 b 是 T 中任意两个结点, 请判断以下结论的正确性。

- (1) 若 a 是 b 的祖先, 则 $\text{post}(a) > \text{post}(b)$;
- (2) 若 a 是 b 的后代, 则 $\text{pre}(a) < \text{pre}(b)$;
- (3) 若 a 是 b 的后代, 则 $\text{in}(a) < \text{in}(b)$;
- (4) 若 a 在 b 的左边, 则 $\text{in}(a) < \text{in}(b)$;

8. 已知一棵二叉树的中序序列和后序序列分别为 $B D C E A F H G$ 和 $D E C B H G F A$, 请画出这棵二叉树。

9. 试分别找出满足以下条件的所有二叉树:

- (1) 二叉树的先序序列与中序序列相同;
- (2) 二叉树的中序序列与后序序列相同;
- (3) 二叉树的先序序列与后序序列相同。

10. 画出与图 10.37 所示树对应的二叉树。

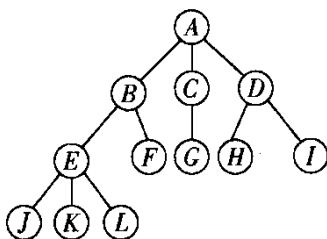


图 10.37 树

11. 画出与图 10.38 所示各二叉树对应的森林。

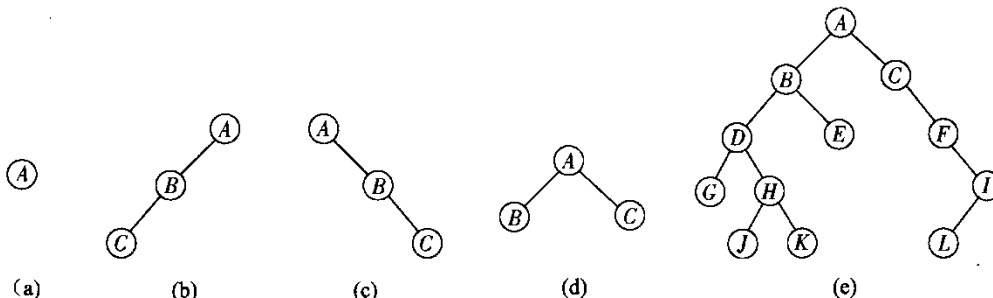


图 10.38 二叉树

12. 若哈夫曼树中有 n 个叶结点, 则该哈夫曼树中共有多少个结点?

13. 给定数值集合 $\{15, 3, 14, 2, 6, 9, 16, 17\}$, 请构造相应的哈夫曼树, 并计算它的带权路径长度。假定用于通信的电文仅由 8 个字母 A、B、C、D、E、F、G、F 组成, 以上数值集合表示了各字母在电文中出现的频率, 请给出这 8 个字母的哈夫曼编码。

14. 对如图 10.39 所示的有向图请给出: